# Implementación del Método de Fase de Acotamiento para Optimización Unidimensional mediante Programación Orientada a Objetos

Kristopher Vela Domínguez
Ingeniería en Inteligencia Artificial
Universidad de Xalapa
Xalapa, Veracruz, México

***Resumen*—En este trabajo se presenta el desarrollo de una herramienta de software orientada a la resolución de problemas de optimización unidimensional utilizando el Método de Fase de Acotamiento. A diferencia de técnicas de búsqueda directa, este algoritmo determina un intervalo acotado que garantiza contener el mínimo global de la función mediante incrementos exponenciales. La implementación se realizó bajo el paradigma de Programación Orientada a Objetos (POO), encapsulando la lógica de búsqueda y permitiendo la inyección de funciones objetivo de forma dinámica. Se evaluaron tres funciones de prueba (exponencial, polinomial y logarítmica) para validar el desempeño del sistema.**

***Index Terms*—Optimización, fase de acotamiento, programación orientada a objetos, Python, búsqueda iterativa.**

## I. Introducción

La optimización de funciones es un problema fundamental en la ingeniería y las ciencias computacionales. En este trabajo se implementa un algoritmo de optimización unidimensional basado en el Método de Fase de Acotamiento, el cual sirve como una etapa heurística inicial para localizar una región o intervalo $[a, b]$ donde reside un punto óptimo.

El objetivo es desarrollar un sistema flexible que permita evaluar distintas funciones sin modificar el núcleo del algoritmo, aplicando principios de Programación Orientada a Objetos y automatizando decisiones lógicas, como la dirección de búsqueda y el crecimiento exponencial del tamaño de paso.

## II. Metodología

### *II-A. Método de Fase de Acotamiento*

El algoritmo inicia en un punto $x_0$ con un incremento base $\Delta$. Primero, evalúa el comportamiento de la curva evaluando la función en $x_0 - |\Delta|$, $x_0$ y $x_0 + |\Delta|$ para determinar la dirección de descenso. Si la función decrece hacia la derecha, el paso toma valor positivo; si decrece a la izquierda, negativo.

Una vez definida la dirección, el método aplica saltos iterativos que crecen de manera exponencial utilizando la siguiente regla de actualización:

$$x^{(k+1)} = x^{(k)} + 2^k \Delta \tag{1}$$

El algoritmo se detiene en la iteración donde el valor de la función comienza a incrementarse, es decir, cuando se cumple la condición:

$$f(x^{(k+1)}) \geq f(x^{(k)}) \tag{2}$$

El mínimo queda entonces encerrado en el intervalo $[x^{(k-1)}, x^{(k+1)}]$.

### *II-B. Diseño orientado a objetos*

Se implementó una clase denominada **FaseAcotamiento**, la cual encapsula mediante atributos privados:

- Función objetivo (inyectada como función lambda)
- Historial de iteraciones
- Nombre de la función

El método principal solicita únicamente el punto de partida y el incremento, manteniendo la lógica matemática aislada y modular.

## III. Implementación

El sistema fue desarrollado en Python utilizando bibliotecas como *NumPy* para cálculos numéricos y *Matplotlib* para la visualización gráfica de los saltos. El algoritmo evalúa el cambio de pendiente de manera autónoma y devuelve el intervalo acotado.

## IV. Resultados

### *IV-A. Función 1: Exponencial*

$$f(x) = e^x - 5x \tag{3}$$

Parámetros iniciales: $x_0 = 0$, $\Delta = 0{,}5$
Resultado:

- Dirección de búsqueda: Positiva ($+\Delta$)
- Intervalo acotado final: $[0{,}5, 3{,}5]$

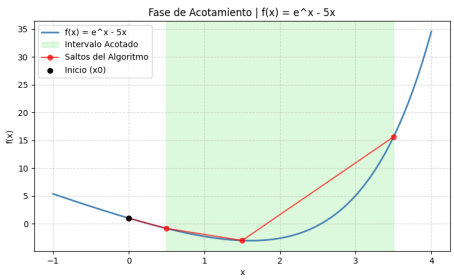


Figura 1. Función exponencial ilustrando los saltos iterativos hasta acotar el mínimo global.

### *IV-B. Función 2: Polinomial*

$$f(x) = x^4 - 3x^3 + 2 \tag{4}$$

Parámetros iniciales: $x_0 = 0$, $\Delta = 0{,}1$
Resultado:

- Dirección de búsqueda: Positiva ($+\Delta$)
- Intervalo acotado final: $[0{,}7, 3{,}1]$

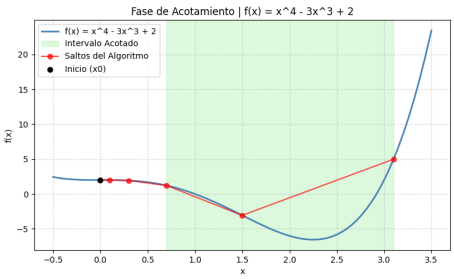


Figura 2. Función polinomial evidenciando el crecimiento del paso de búsqueda de forma exponencial.

### *IV-C. Función 3: Logarítmica*

$$f(x) = x^2 - \ln(x) \tag{5}$$

Parámetros iniciales: $x_0 = 0{,}5$, $\Delta = 0{,}2$
Resultado:

- Dirección de búsqueda: Positiva ($+\Delta$)
- Intervalo acotado final: $[0{,}5, 1{,}1]$

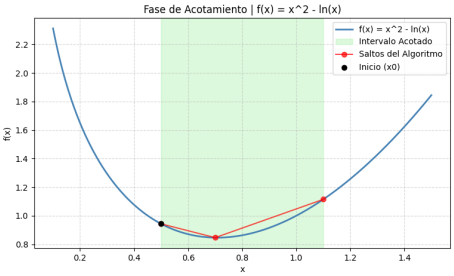


Figura 3. Función logarítmica demostrando la rápida convergencia del algoritmo en tan solo dos iteraciones.

## V. Discusión

Los resultados demuestran que el Método de Fase de Acotamiento es notablemente más eficiente que la búsqueda exhaustiva para encerrar un mínimo global. Gracias al crecimiento exponencial $2^k$, el algoritmo abarca grandes secciones del dominio en muy pocas iteraciones.

El uso de POO permitió desarrollar una solución modular que detecta la dirección de búsqueda dinámicamente y puede integrarse fácilmente con otros algoritmos de refinamiento, como la Búsqueda de la Sección Áurea.

## VI. Conclusión

Se implementó exitosamente el algoritmo de Fase de Acotamiento utilizando programación orientada a objetos. El sistema evaluó correctamente la pendiente inicial, aplicó la expansión exponencial de manera exacta y logró encapsular el mínimo global en los intervalos calculados para las tres funciones solicitadas.

## VII. Referencias

### Referencias

[1] S. Chapra, *Métodos Numéricos para Ingenieros*. McGraw-Hill.
[2] K. Deb, *Optimization for Engineering Design: Algorithms and Examples*. PHI Learning Pvt. Ltd.
[3] Python Software Foundation, “Python Documentation”.